Future Generation Computer Systems 182 (2026) 108452



Contents lists available at ScienceDirect

Future Generation Computer Systems

journal homepage: www.elsevier.com/locate/fgcs



Morpheus: Lightweight RTT prediction for performance-aware load balancing

Panagiotis Giannakopoulos [a,*], Bart van Knippenberg [b], Kishor Chandra Joshi [a], Nicola Calabretta [a], George Exarchakos [a]

[a] Eindhoven University of Technology, Eindhoven, The Netherlands
[b] Thermo Fisher Scientific, Eindhoven, The Netherlands

ARTICLE INFO

Keywords:
Edge computing
Performance variability
Performance predictability
Monitoring metrics
Kubernetes
Prometheus

ABSTRACT

Distributed applications increasingly demand low end-to-end latency, especially in edge environments where co-located workloads contend for limited resources. Traditional load-balancing strategies are typically reactive and rely on outdated or coarse-grained metrics, often leading to suboptimal routing decisions and increased tail latencies. This paper presents a methodology for building and evaluating round-trip time (RTT) predictors that enable proactive and performance-aware load balancing. We develop fast, lightweight, and accurate RTT predictors that are trained on time-series monitoring data collected from a Kubernetes-managed GPU cluster. By leveraging a reduced set of highly correlated monitoring metrics, our approach maintains low overhead while remaining adaptable to diverse co-location scenarios and heterogeneous hardware. The predictors achieve up to 95% accuracy while keeping the prediction delay within 10% of the application RTT. In addition, we analyze the main contributing factors to prediction delay and provide insights for deploying effective predictors in resource-constrained clusters. Simulation-based analysis illustrates how these predictors can guide load-balancing decisions to reduce application RTT and resource waste, paving the way for their integration into real-time cluster management frameworks.

1. Introduction

Modern distributed applications, including web microservices, real-time streaming, and Open RAN workloads, require low end-to-end latency [1]. In such systems, load balancers play a key role by routing requests across application replicas, directly affecting the user experience. Traditional strategies rely on aggregate metrics (e.g. CPU load) sampled at fixed intervals. Although easy to implement, these reactive methods often use outdated data, leading to suboptimal routing, degraded performance, and increased tail latency [2].

To reduce operational costs, applications are frequently co-located in edge and cloud environments, which intensifies resource contention and execution time variability [3]. Diverse hardware and software stacks further exacerbate performance fluctuations across nodes [4,5]. Thus, achieving predictable performance requires careful resource management [6], efficient utilization [7], and scale automation.

Recent advances in machine learning and time-series analysis have enabled accurate round-trip time (RTT) prediction using system monitoring data such as CPU utilization [8] or hardware characteristics [9]. However, most approaches depend on fixed, coarse-grained metrics and do not adapt to co-location dynamics or hardware heterogeneity. Furthermore, accurate prediction models often require complex retraining pipelines that introduce overhead, making them impractical for high-speed load balancers. Therefore, integrating predictive models while maintaining throughput and responsiveness remains a challenge.

To address these challenges, we study performance predictability within an electron microscopy (EM) workflow, focusing on the Single Particle Analysis (SPA) pipeline. SPA tasks operate under sub-second latency constraints and demand diverse resources (such as CPU, GPU, network). Deploying them on shared, distributed infrastructure enables dynamic resource pooling across multiple EM systems, improving utilization. In this context, proactive, prediction-aware load balancing mitigates the limitations of reactive schemes by anticipating per-request latency before routing and reducing tail delays.

We present Morpheus, a framework for predicting RTT per-request on sub-second timescales using a carefully selected subset of system-state metrics that reflect both compute and network conditions. Experiments are performed on a Kubernetes GPU cluster with Prometheus-based monitoring at 200 ms intervals. Predictors run alongside applications, evaluating multiple models to select the one that optimally

* Corresponding author.
E-mail address: p.giannakopoulos@tue.nl (P. Giannakopoulos).

https://doi.org/10.1016/j.future.2026.108452
Received 1 August 2025; Received in revised form 9 December 2025; Accepted 21 February 2026
Available online 2 March 2026
0167-739X/© 2026 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

balances accuracy and inference time. By operating under tight timing constraints and relying only on the most informative metrics, Morpheus delivers low-latency predictions suitable for integration with real-time load balancers.

We evaluate Morpheus in a real runtime system, measuring prediction accuracy, overhead, and delay across heterogeneous servers. The results show high accuracy (up to 95%) with minimal overhead (below 10% of the RTT) and identify bottlenecks that limit the prediction speed. Simulation studies based on real cluster traces further quantify how predictor accuracy influences load-balancing efficiency, showing clear performance improvements. The main contributions are:

- A practical methodology for designing RTT predictors that operate in runtime and learn application-specific performance behavior.
- A simulation study quantifying performance-aware load balancing under varying predictor accuracy, application replica counts, and cluster heterogeneity.
- Insights to improve predictor efficiency and robustness in heterogeneous edge clusters.

The paper is organized as follows: Section 2 reviews related work; Section 3 presents the architecture; Section 4 describes the setup; Sections 5 and 6 discuss the prediction and simulation results; Section 7 outlines the lessons learned; and Section 8 concludes.

2. Related work

Applications in resource clusters often experience variable performance due to shared resources, hardware aging, and frequent system updates [10,11]. Prior studies have examined variability across operating systems, hardware, and deployment environments. For example, Chunduri et al. [12] analyzed the role of the operating system and hardware configuration, while Maricq et al. [4] developed CONFIRM to assess the variability in subsystems such as memory and I/O. Duplyakin et al. [5] extended this line of work by modeling CPU-level heterogeneity. Other works have examined the effects of code changes [13], co-location [3], and software environments [14]. Although observability tools [15–17] and trace-based diagnostics [18–20] help to detect anomalies post hoc, they lack predictive capability. Our work incorporates these sources of variability into lightweight runtime predictors to enable proactive decision-making at the request-level.

Learning-based performance prediction has gained significant traction in recent years. Zhao et al. [21] used gradient-boosted trees to detect performance regressions caused by system updates, but their method does not capture co-location dynamics. Mohamed et al. [8] employed SHAP [22] to interpret predictions from a limited fixed metric set, although the high computational cost restricts real-time use. Instead, we adopt perfCorrelate [23], a lightweight method for identifying predictive monitoring metrics at runtime. Unlike progress-based approaches that require application instrumentation [6], our method relies solely on system-level monitoring data already available in production clusters to forecast RTT before execution. Deployment-time frameworks such as Quasar [24], Dauwe et al. [25], and Townend et al. [7] model application behavior to guide placement, but do not support request-level routing. Li et al. [26] use DRL to optimize task migration, and Gao et al. [27] apply LSTM-based prediction to manage cloud gaming resources, yet both operate at workflow or session granularity and rely mainly on aggregated network statistics. Our work instead leverages instantaneous compute and network metrics to predict RTT at the request level, enabling fine-grained, latency-aware load balancing.

Traditional load balancing methods, such as round-robin and least-connections [28,29], are static and cannot react to rapid performance changes. Adaptive and context-aware schemes [30–32] adjust routing based on recent history or offline profiles, but remain fundamentally reactive. Prequal [2] probes servers to estimate near-term response quality, yet cannot anticipate degradation. Distributed and collaborative mechanisms such as BLOC [33] and federated ML schedulers [34] exploit runtime signals or jointly trained models, but introduce coordination overhead that limits per-request responsiveness. Hu et al. [35] introduced LACC, a DRL-based congestion control scheme that selects transport-layer mechanisms under dynamic conditions, while Hu et al. [36] proposed DRLB, a DRL-driven load balancer that differentiates long and short flows using global traffic statistics. Both approaches focus on network-layer congestion and flow-level decisions, whereas our method predicts RTT using instantaneous compute and network metrics at the request level to support fine-grained and performance-aware load balancing.

In summary, although previous work has advanced anomaly detection, scheduling, and reactive adaptation, real-time, per-request RTT prediction for proactive, latency-aware load balancing remains underexplored. Our work addresses this gap by developing fast, low-overhead RTT predictors that use live monitoring signals and are suitable for integration into fine-grained request steering in time-sensitive environments.

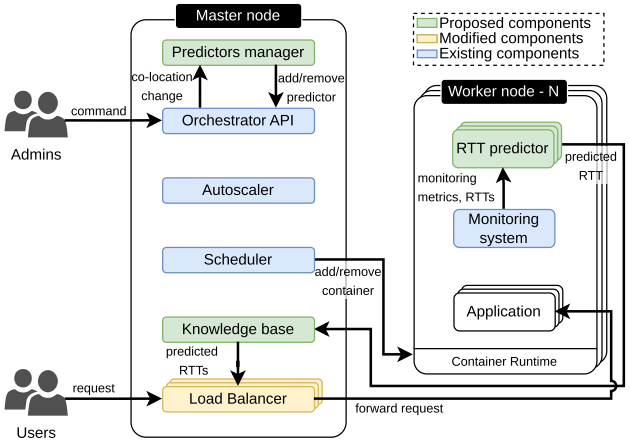


Fig. 1. High-level architecture for integrating RTT predictors into the load balancer of a resource cluster.

3. Predictor architecture

The proposed run-time predictors operate alongside the target application and continuously learn its performance behavior over time. Fig. 1 provides a high-level overview of how RTT predictors are integrated into the load balancer. Application performance can vary significantly between different server configurations, such as due to differences in CPU models [4,5,12]. Therefore, a separate predictor is deployed for each unique combination of application and server within the cluster. Each predictor collects RTT measurements along with the corresponding monitoring metrics for every task execution using the monitoring system. These data are used to train and update machine learning models that estimate the RTT for future tasks. In this context, RTT is defined as the time interval between when a request enters the cluster gateway (t_{start}) and when the corresponding response exits the gateway (t_{end}). This definition captures in-cluster performance by excluding network transmission delays between the client and the gateway. Each task corresponds to a single request-response cycle initiated by the user.

Predictions are stored in a database called the knowledge base and later used by each application load balancer to guide placement decisions. The Prediction Manager monitors application activity within the cluster, deploying an RTT predictor when an application runs on a server for the first time, re-enabling it if previously deployed, and pausing it when all replicas are removed. To ensure accurate RTT prediction for new tasks, Morpheus predictors run three parallel processes: data collection, training, and prediction, as shown in Fig. 2.

Data Collection: This process collects RTT and monitoring metrics for each completed task using the monitoring system. For each task, it collects monitoring data from the period just before task submission (referred to as observation window), along with the corresponding RTT value. Correlation analysis, through perfCorrelate [23], is performed to evaluate the relationship between each monitoring metric and the RTT. As demonstrated in [23], this correlation-based methodology identifies and provides a deeper understanding of the key factors at the system-level that influence performance variability, showing how resource contention and node-level conditions affect RTT. This is done using several correlation algorithms and different observation windows. As a result, the process identifies both the most relevant metrics and the best observation window to capture system state. The size of the retrieved system state is determined by the number of selected metrics and the length of the observation window. To ensure that the system state does not introduce unacceptable delays during predictions, a strict time limit is imposed on its retrieval. The dataset is stored in two forms: raw time-series data and feature-based representations, such as mean or standard deviation, to support different types of models.

Training: As the dataset is updated, the process creates or updates machine learning models using the latest monitoring metrics and RTTs. These metrics are prepared in two forms to accommodate different types of models. For models specialized in time-series data, such as recurrent neural networks, the raw time-series of monitoring metrics are used as input. For models that do not require sequential data, such as XGBoost, statistical features (for example, mean and standard deviation) are extracted from each monitoring metric. The process evaluates various model types with both data representations and selects the one that provides the highest prediction accuracy while meeting the required inference time constraints.

Prediction: For each new task, the process retrieves the required recent data from the system state, extracts features if necessary, and uses the model to estimate the RTT. The predicted RTT is then stored in the knowledge base for later use by the load balancer.

3.1. Data collection process

The data collection process runs at regular intervals, repeating every five minutes. Each cycle includes a sequence of steps that correspond to the components shown in the green panel of Fig. 2. The steps are the following:

New data check: The process checks whether the monitoring system has recorded new task records since the last cycle. If no new data are found, the process enters a sleep state until the next scheduled run.

RTT collection: If new tasks exist, they are retrieved from the monitoring system. Specifically, for each task, the (t_{start}), (t_{end}) and RTT are stored.

Balance RTT data: In practice, tasks can arrive at a high rate, causing the dataset to grow rapidly as new data are collected. When the data distribution is skewed, many of these new samples can be duplicates or highly similar to existing ones, leading to unnecessary redundancy in the dataset. Storing all task records and loading them into memory becomes impractical, particularly for servers with limited resources. This challenge is even more pronounced when dealing with resource usage monitoring metrics. For example, if we store 60 s of historical data prior to each task submission for 500 metrics, with each metric sampled at 5 values per second (i.e., every 200 ms), this results in $500 \times 5 \times 60 = 150{,}000$ data points per task.

Additionally, if the RTT distribution in the dataset is non-uniform, the model may become biased toward the most common RTT values, which reduces prediction accuracy [37]. To address both memory limitations and the risk of model bias, we construct a compact training data set that preserves the RTT distribution as uniformly as possible. This is achieved using a dynamic binning strategy, which selectively retains only those samples that help preserve a balanced RTT distribution. The details of how this strategy operates depend on whether an initial dataset is already present:

Case 1 (no existing training data): In the initial phase, when no prior dataset exists, all collected RTT samples are retained without any filtering or subsampling. This approach ensures that potentially valuable information is not discarded at the outset, preserving the complete diversity and variability of the observed RTT values. Keeping the full set of initial data is especially practical in this context because data acquisition occurs at a moderate frequency (e.g. every 5 min), which means that the dataset size remains small and manageable during early collection periods. By retaining all initial samples, the method provides a comprehensive and unbiased foundation for defining future dataset structure and balancing criteria as new data become available.

Case 2 (existing training data): When new RTT samples arrive, the bin boundaries are recalculated using the combined RTT values from both the existing dataset and the new samples. Specifically, the set S consists of all RTT values from both sources, and the bin width is calculated using the Freedman–Diaconis rule [38]:

$$h = \frac{2 \cdot \mathrm{IQR}(S)}{N^{1/3}}, \tag{1}$$

where $\mathrm{IQR}(S) = Q_{75}(S) - Q_{25}(S)$ and N is the total number of RTT samples in S. The number of bins l and their upper boundaries b_i are then:

$$l = \left\lceil \frac{\max(S) - \min(S)}{h} \right\rceil, \quad b_i = \min(S) + i \cdot h, \quad i = 1 \cdots l \tag{2}$$

Each new RTT sample is then assigned to a bin and retained only if the number of existing samples in that bin is less than the maximum bin count, $C_{\max}$, across all bins. If more new samples arrive for a bin than its available capacity, a random subset of new samples is selected to fill the gap. The remaining capacity for a bin i is given by:

$$\mathrm{gap}_i = C_{\max} - c_i, \tag{3}$$

where c_i is the existing sample count in the bin i. New samples are added to their bin up to this available capacity. If no bin has remaining capacity for any new sample, a single new sample is retained at random to ensure the dataset can continue to evolve.

In principle, the uniformity of the RTT distribution could be further improved by removing samples from the overrepresented bins in the existing dataset. However, this would also require the removal of the associated monitoring metrics for each RTT sample. Experimental measurements show that each RTT entry is about 77B, while the corresponding metrics total over 500kB due to 294 metric lines of roughly 1.8kB each. Because the monitoring dataset is orders of magnitude larger, removing samples would require extensive coordination and significantly increase both memory usage and processing time. Therefore, the proposed method focuses on the selective addition of new samples, providing a practical balance between improving uniformity and minimizing overhead.

Metrics collection: For each new task in the balanced dataset, the process retrieves the monitoring metrics from a 60 s window that precedes each task submission. This window is chosen because it is long enough to capture short-term fluctuations and resource contention that can affect task performance, yet short enough to avoid including outdated or irrelevant system states.

Dataset size check: The process assesses whether the existing dataset has sufficient data points to capture performance variability. This is done through CONFIRM [4], which estimates the minimum number of repetitions needed to ensure that the median RTT is within $r\%$ of the true median with confidence $\alpha\%$. This method is robust for non-normally distributed data [39]. If the dataset contains a sufficient number of samples, the process continues with the subsequent steps. Otherwise, it skips the remaining steps for the current cycle and waits until the next one.

Correlations check: This step checks whether the previously established correlations between monitoring metrics and RTT remain valid. Correlations are considered valid if they exist and no notification has

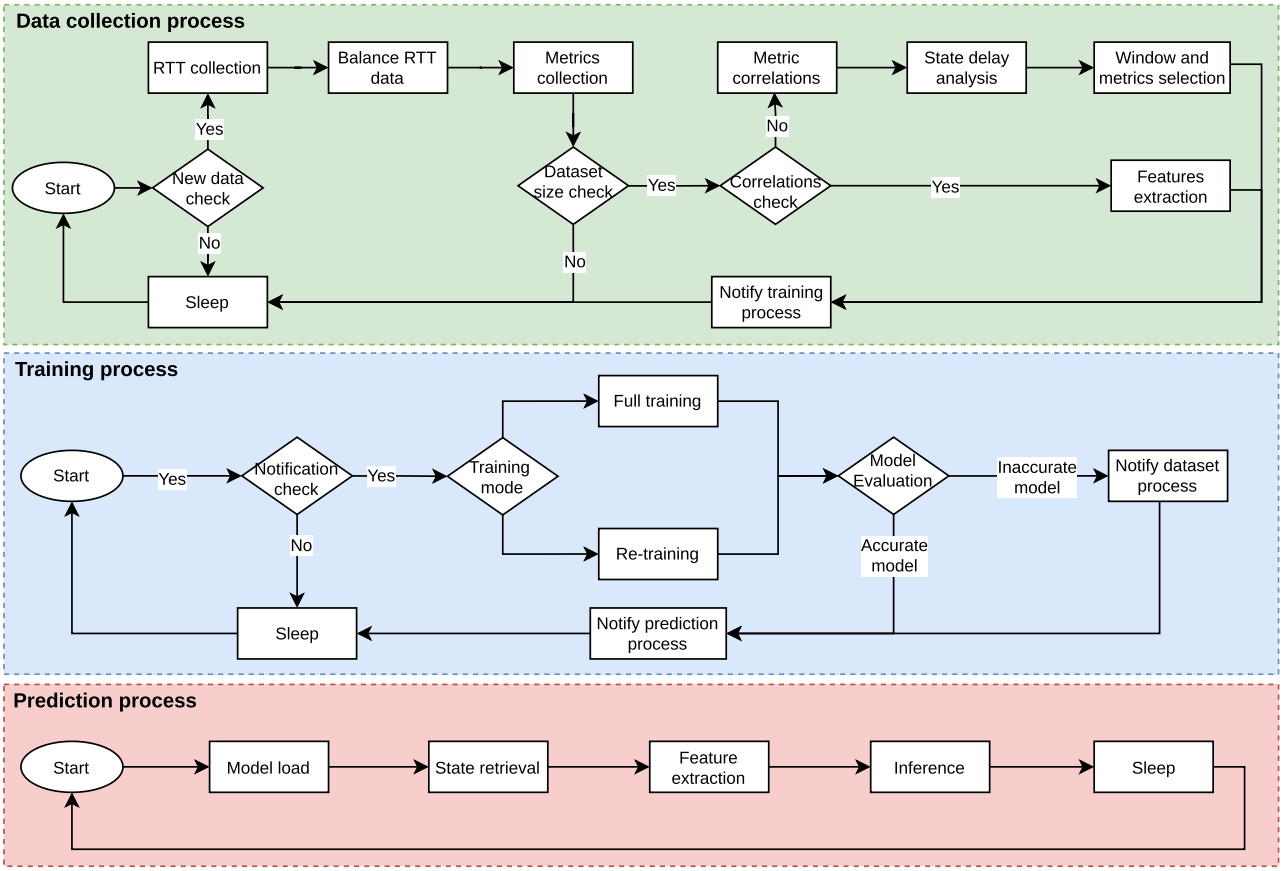


Fig. 2. Overview of the RTT prediction methodology.

Table 1
Correlation methods used in *perfCorrelate*.

Method	Range	Description
Pearson	$[-1, 1]$	Measures linear correlation between continuous variables.
Spearman	$[-1, 1]$	Detects monotonic relationships using rank order of data.
Kendall	$[-1, 1]$	Measures ordinal association based on concordant and discordant pairs.
Distance	$[0, 1]$	Quantifies general dependence using pairwise distances between samples.
MIC	$[0, 1]$	Identifies linear and nonlinear dependencies using mutual information.

been issued by the training process (more details in Section 3.2). If the correlations are valid, the process jumps to the *feature extraction* step.

Metric correlations: The correlation analysis is performed by invoking the `perfCorrelate` module [23]. This module operates in three stages: (1) it extracts, for each monitoring metric in the form of a time-series, the most relevant statistical feature using `tsfresh` [40], selecting the one with the highest correlation with RTT, (2) it eliminates redundant metrics to reduce dimensionality, and (3) it computes correlations between each remaining metric and RTT using multiple methods. The used correlations are Pearson, Spearman, Kendall, Distance Correlation, and Maximum Information Coefficient (MIC). An overview of the correlation methods supported is provided in Table 1. The absolute values of correlation scores are used to assess the relevance of each metric. This approach ensures that, regardless of the correlation method, all scores are in the range between 0 and 1. The monitoring metrics values are correlated with RTT using observation windows of 1, 5, 20, and 60 s before the task is submitted.

For each monitoring metric, the correlations are calculated using all available methods. The method that yields the highest absolute correlation score for a given metric is selected as its representative. This procedure is applied independently across all observation windows, resulting in a comprehensive set of correlation results indexed by metric, selected correlation method, and observation window.

State delay analysis: The overhead associated with retrieving time-series data from the monitoring system and extracting features using `tsfresh` is then measured. This includes the time required for state retrieval (t^k_{state}) and features calculation (t^k_{feature}) for each number k of monitoring metrics. The process begins with the five most correlated metrics and incrementally adds the next five most correlated metrics in each step, continuing this procedure until all relevant features have been included.

Window and metrics selection: The optimal combination of the observation window w, the correlation method r, and the number of metrics k is selected to balance predictive accuracy with the overhead of preparing the input, which includes both the retrieval of the state of the system and the extraction of features. Among all evaluated combinations of w, r and k, only those that satisfy the predefined overhead constraint are considered for selection:

$$t^k_{\text{state}} + t^k_{\text{feature}} \leq \tau_{\text{prepare}} \cdot \mu_{\text{RTT}} \tag{4}$$

where τ_{prepare} defines the maximum allowed fraction of the average RTT (μ_{RTT}) that can be spent on input preparation. If no combination meets this condition, no predictor is trained for that application–node pair. In our work, we use $\tau_{\text{prepare}} = 9\%$, empirically calibrated to ensure that the combined input-preparation and inference overhead remains within 10% of the mean RTT, thus preserving near-real-time responsiveness.

Table 2
Compact overview of suitable ML models based on correlation type and dataset size.

Correlation Type	Dataset Size	Models	Justification
Pearson (Linear)	Any size	LR, XGB	Models linear relations efficiently. LR and XGB generalize well across dataset sizes with low complexity.
Spearman (Monotonic)	Any size	RF, XGB, SVM	Effective for monotonic trends. RF and XGB handle ordinal data; SVM captures non-linear boundaries.
Kendall (Monotonic)	Any size	RF, XGB	Similar to Spearman; RF and XGB model rank-based relations robustly and remain interpretable.
Distance / MIC (Non-linear)	Small (<1K)	XGB	XGB captures complex non-linear patterns in small datasets with minimal risk of overfitting.
	Medium (1K–10K)	XGB, FNN	Both XGB and shallow FNNs handle moderate-size data well, balancing complexity and accuracy.
	Large (>10K)	XGB, FNN, RNN, CNN	RNNs/CNNs learn intricate patterns; FNN and XGB still offer strong performance and efficiency.

The optimal combination of w, r and k, denoted by an asterisk, is the one that maximizes the total correlation with RTT:

$$(w^*, r^*, k^*) = \arg\max_{w,r,k} \left\{ \sum_{i=1}^{k} |\rho_{w,r}(m_i, \text{RTT})| \right\}, \tag{5}$$

where $\rho_{w,r}(m_i, \text{RTT})$ is the correlation between the monitoring metric m_i and the RTT, calculated in the observation window w using the correlation method r.

Feature extraction: In this step, the monitoring metrics from newly collected samples are transformed into statistical features and added to the dataset. This transformation is only performed if the metric correlations have not already been computed during the current cycle, as the correlation analysis step also handles feature extraction and dataset updates. Feature extraction is triggered only if required by the training process, specifically when the model expects input in the form of aggregated features rather than time-series data (i.e., non-sequential format).

Training notification: Once the dataset is updated by either the *Metrics Collection* or *Feature Extraction* step, the training process is notified. After this notification, the data collection process is paused and waiting until the next cycle begins.

3.2. Model training process

The model training process follows an event-driven manner, initiated whenever the model needs to be created or updated. This can occur either when new data arrive or when a decrease in the accuracy of the current model is detected. It consists of a sequence of steps corresponding to the components in the blue panel of Fig. 2. These steps are as follows:

Training mode: The process chooses between two training modes: (1) *full training*, where multiple model types are trained from scratch to find the best-performing model, and (2) *re-training*, where the current model is updated with newly collected data. *Full training* is used when there is no existing model or when the current model is no longer accurate enough. Otherwise, if the existing model still performs well, *re-training* is performed.

Full training: In full training, a broad set of candidate models is trained from scratch and evaluated. We categorize performance prediction models into two distinct groups: non-sequential and sequential models. This classification is based on the nature of the input data. Non-sequential models are suitable for structured datasets where the order of observations does not influence the outcome, while sequential models are designed to capture temporal dependencies and evolving patterns over time. This dual approach allows us to flexibly address both static and time-dependent behaviors in predicting application performance.

Each model in the respective categories receives input data X, which represents the monitoring metrics either as extracted features or as time-series data, and produces output Y, the predicted RTT for the target application. Both X and Y are normalized using MinMax scaling in a range between 0 and 1. Outliers are removed from the dataset (z-score > 3) before further processing. The data is then divided into three subsets: 80% for training, 10% for validation, and 10% for testing. The training set is used to fit the models, the validation set to tune hyperparameters, and the test set to assess the final performance of the models.

Non-sequential models are widely used for analyzing tabular data, where each sample is represented as a fixed-length vector and treated independently of temporal order. In our methodology, this category includes Linear Regression [41], XGBoost [42], Random Forest [43], and Feedforward Neural Networks (FNN) [44], all of which are well-suited for learning from aggregated monitoring metrics and statistical features rather than raw time-series inputs. These models use a feature vector $X = (x_1, x_2, \ldots, x_M)$, where each x_i corresponds to a feature extracted from the monitoring metrics M using `tsfresh`, based on the selected observation window.

Sequential models are tailored for scenarios where the order and timing of data points carry meaningful information. This category includes Recurrent Neural Networks (RNN) [45], Long Short-Term Memory (LSTM) [46], Gated Recurrent Units (GRU) [47] and Convolutional Neural Networks (CNN) [48]. These models are effective in handling time-series data and can learn complex temporal patterns and long-term dependencies. The input to these models is a sequence $\mathbf{X} = (X_1, X_2, \ldots, X_M)$, where each X_i represents the full time-series of a specific monitoring metric within the observation window.

The selection of candidate machine learning models can be guided by the type of correlation r^* between the features of the monitoring metrics [49] and the target variable, as well as the size N of the dataset [50]. Table 2 presents the recommended model options for each correlation type and data size. By following these recommendations, only a selected subset of models is evaluated, which significantly reduces overall training time.

Each candidate model is evaluated on the basis of two criteria: Root Mean Squared Error (RMSE) and inference time. Only models whose inference time remains below a defined fraction $\tau_{\text{inference}}$ of the mean RTT of the application are considered acceptable. In our work, we use $\tau_{\text{inference}} = 1\%$, chosen through empirical measurement so that, together with the preparation phase, the overall prediction delay does not exceed 10% of the application RTT. Among them, the model with the lowest RMSE is selected as optimal. The optimization problem is defined as follows:

$$f_{\text{select}} = \arg\min_{ml} \text{RMSE}(ml), \quad \text{subject to} \quad t_{\text{inference}}(ml) \leq \mu_{\text{RTT}} \cdot \tau_{\text{inference}} \tag{6}$$

In this formulation, ml denotes a candidate machine learning model trained on input features k^* extracted over the observation window w^*. The RMSE of the model is indicated by $\text{RMSE}(ml)$, and $t_{\text{inference}}(ml)$ is the inference time. μ_{RTT} represents the mean RTT of tasks towards the target application.

Re-training: The re-training strategy depends on the model type. For sequential models and FNN, online training is used to incrementally update the model as new data become available, enabling runtime adaptation without requiring full training. For all other non-sequential models (e.g., LR, XGBoost), the model is retrained on the entire dataset while retaining the previously tuned hyperparameters.

Model Evaluation: Each newly re-trained model is evaluated by comparing its prediction error with that of the previous model, if one exists. Specifically, the training process calculates the relative change in RMSE between the new and the previous models as follows:

$$\mathrm{RMSE}_{\mathrm{change}} = \frac{\mathrm{RMSE}_{\mathrm{new}} - \mathrm{RMSE}_{\mathrm{prev}}}{\mathrm{RMSE}_{\mathrm{prev}}} \tag{7}$$

If this change exceeds a predefined threshold θ, the data collection process is notified to re-evaluate the correlations between the monitoring metrics and the RTT. Based on updated correlations, the input dataset is revised. Once this revision is complete, the training process is triggered to train a model from scratch (full training). In our experiments, we set the threshold to $\theta = 10\%$, which was empirically found to balance model stability and adaptability, triggering retraining only when prediction accuracy drops significantly without causing unnecessary updates. These parameters (τ_{prepare}, $\tau_{\mathrm{inference}}$, and θ) must be tuned for the target production system and application-specific SLA to balance prediction accuracy, overhead, and full training frequency.

Prediction process notification: Once a model is created or updated, the prediction process is notified. After this notification, the training process is paused and waited until a new notification is received.

3.3. Prediction

The process can be configured to operate either in an on-demand mode triggered through HTTP requests or in a periodic mode where predictions are made at regular intervals. In on-demand mode, predictions are always fresh but can cause higher overhead under high request rates, since a prediction is computed for each request. In periodic mode, predictions are precomputed and reused, reducing overhead during high load but potentially becoming slightly outdated between updates. The prediction process consists of a sequence of steps corresponding to the components in the red panel of Fig. 2. These steps are as follows:

Model load: Upon notification, the newly trained model is loaded and ready for predictions.

State retrieval: The process retrieves the required monitoring metrics, specifically, the top metrics k^* within the optimal observation window w^*, determined by the data collection process, from the monitoring system.

Feature extraction: Depending on the model type, it may also extract time-series features from the retrieved monitoring metrics through tsfresh.

Inference: The model performs an inference to estimate the RTT of the upcoming task. The predicted value is stored in a database (referred to as knowledge base) for subsequent analysis. The prediction delay can be divided as follows:

$$t_{\mathrm{prediction}} = t_{\mathrm{state}} + t_{\mathrm{feature}} + t_{\mathrm{inference}} \tag{8}$$

4. Experimental setup

This section presents the applications, software frameworks, compute infrastructure, and experimental workflows deployed.

4.1. Applications

Our experiments employ five SPA applications: *Upload*, *MotionCor2*, *FFT Mock*, *gCTF*, and *ctffind4*. Each application receives a POST request with an input configuration, processes the request using a specific algorithm, and returns the result. Applications are single-threaded and handle one task at a time. Their functions are as follows:

- **Upload:** Transfers an image file from the client to the target worker node.
- **MotionCor2:** Compensates for drift and beam-induced motion during image acquisition, improving image quality [51,52]. It merges multiple image frames into a corrected single-frame image (MRC format [53]). Version 1.5.0 is used.
- **FFT Mock:** Emulates lightweight image processing workloads by performing pixel duplication, shifting, and Fast Fourier Transform (FFT)-based operations to simulate proprietary algorithms.

Table 3
Specifications of infrastructure components. Worker-1 is equipped with a Tesla K20c GPU, Worker-2 with a Tesla M40, and Worker-3 with an RTX 4090. All other nodes do not have a GPU. Workers 1, 2, 5, and 9–29 are equipped with HDD, while Workers 3, 4, 6, 7, and 8 use SSD.

Name	Model/Processor	Cores	RAM
Switch	Dell PowerConnect 5448	–	–
Routing	Intel Xeon X3430	3	4GB
Master	Intel Xeon Silver 4109T	8	32GB
Monitor	Intel Xeon Silver 4109T	8	78GB
Worker-1	Intel Core i7-7700	4	32GB
Worker-2	2xIntel Xeon E5-2637	4	24GB
Worker-3	Intel Core i9-14900K	32	128GB
Worker-4	Intel Core i5-9600	4	12GB
Worker-5	2xIntel Xeon E5504	8	24GB
Worker-6	Intel Xeon Silver 4209T	8	24GB
Worker-7	Intel Xeon Gold 6138T	16	64GB
Worker-8	Intel Xeon Silver 4209T	8	24GB
Worker-9-29	Intel Xeon X3430	4	4/8GB

- **gCTF:** Performs GPU-accelerated real-time Contrast Transfer Function (CTF) estimation and correction [54], enhancing image quality for downstream analysis. We use version 1.18.
- **ctffind4:** A CPU-based tool (version 4.1.14) for CTF estimation [55].

Input data are acquired from a vitrified EM grid containing a protein sample, using a Glacios electron microscope equipped with a Falcon4i detector and EPU software [56]. All applications operate on MRC files, with fraction images used by *Upload* and *MotionCor2* and single integrated images by the remaining applications. The fraction files are 897MB each and the integrated files are 65MB. The dataset consists of 98 images.

4.2. Frameworks

We use Kubernetes [57], an open-source container orchestration platform, to manage distributed resources and deploy applications as pods across nodes. Each pod runs one or more containers using container runtimes such as Docker [58] or containerd [59]. The master node manages the scheduling of the pod, while the worker nodes provide the actual resources of the cluster. GPU scheduling becomes available through the installation of GPU plugins.

Monitoring is handled by Prometheus [60], a time-series data collection system integrated with Kubernetes. The Prometheus server scrapes metrics from exporters running on nodes and containers at 200ms intervals and stores the data in a local database. Users can visualize metrics through Grafana [61], its web interface, or its HTTP API. GPU metrics are retrieved using the NVIDIA DCGM exporter [62].

Jaeger [63] is deployed for distributed tracing, tracking end-to-end request latencies across services. It captures RTT within the cluster, but excludes external delays prior to the gateway.

4.3. Compute infrastructure

The experimental cluster (see Fig. 3) includes 32 interconnected servers (see Table 3) linked by 1 Gbps Ethernet. The Kubernetes cluster comprises one master and 30 worker nodes. As performance predictors are tailored per node, the overall cluster size does not impact model accuracy. The node roles are as follows:

- **Routing:** Manages internal network traffic. Not part of the Kubernetes cluster.
- **Master:** Hosts Kubernetes control components and the Docker image registry.
- **Monitor:** Runs MongoDB, Jaeger, Experiment Manager, and Prediction Manager.

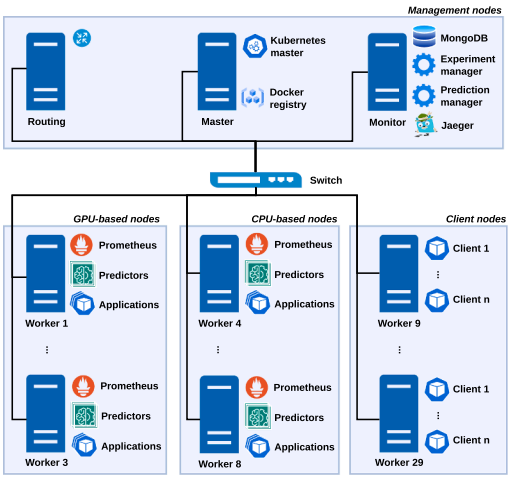


Fig. 3. Cluster architecture. All nodes are interconnected via 1 Gbps Ethernet.

- **Workers:** Workers 1–8 host applications, with Workers 1–3 specifically supporting GPU workloads. Each of these workers (1–8) also hosts its own Prometheus servers and exporters to minimize state retrieval time and eliminate network overhead during state retrieval. In addition, they host RTT predictors for the applications running on those nodes. Workers 9–29 function as clients, generating requests.

4.4. Experimental workflow

In our experimental setup, we deploy as many application replicas as possible in Workers 1 to 8, until the available resources on each node are fully utilized. Resource allocation is unrestricted: applications use minimal resources while idle but can utilize as many resources as needed during active execution. To systematically study the effects of resource contention, we gradually increase the number of co-located application replicas on each node until the node becomes saturated. Once full contention is reached, we progressively reduce the load by stopping task generation for individual applications. To ensure clarity and reproducibility, the experiment is structured into distinct workload stages, each representing a specific level of resource contention.

A *workload stage* is defined as a period during which a specific subset of application replicas is active. Each application has a corresponding client pod that sends a request, waits for the response, and then pauses for a random interval $t_{\text{wait}} \sim \mathcal{U}(0, t_{\max})$ (i.e. uniformly sampled from the interval $[0, t_{\max}]$) before submitting the next request. The maximum waiting times, $t_{\max}$, are set at 40, 6, 20, 10 and 10 s for *upload*, *ctffind4*, *FFT Mock*, *gCTF* and *MotionCor2*, respectively. These values were determined through experiments to ensure that the applications produce tasks that are frequent enough to induce resource contention.

The *Experiment Manager* is deployed as a pod and coordinates the execution of the experiment. It uses the Kubernetes API to deploy application and client pods across all worker nodes. A separate process is initiated per node to activate the initial workload stage. In total, we run 15 workload stages, each lasting a random duration between 11 and 12 hours. Once this condition is met, the next workload stage is activated by enabling more application replicas. After the final stage has completed, all application and client pods are removed.

The resources allocated to each deployed RTT predictor are limited by assigning a maximum of 10% of the node CPU, with an upper limit of one CPU core per predictor. Restricting the CPU allocation in this way helps minimize interference with the applications running on the node. However, this limitation can also slow down the prediction processes, especially model training.

The RTT predictors learn the relationship between monitoring metrics and observed RTTs. For accurate training, especially at deployment, they require diverse RTT values typically caused by resource contention. To induce this variability, the Prediction Manager temporarily adds controlled load: it deploys a *noisy server* pod on the target node and a *noisy client* pod on a separate worker node. The client generates light network traffic to the server, which simulates activity across key system resources: CPU, GPU, RAM, disk, and network, thus inducing controlled interference. Once all RTT predictors on the node establish the correlations between metrics and RTT, the noisy server and client are removed. RTT predictors generate predictions at regular intervals, every 5 s, in order to measure end-to-end prediction latency and detect any performance bottlenecks within the prediction workflow.

5. Evaluation of predictors

In this section, we present the accuracy and overhead of our RTT predictors.

5.1. Selected correlation algorithms

Fig. 4 shows how effective the five correlation algorithms are in capturing the relationship between RTT and monitoring metrics. The algorithms compared are Pearson, Spearman, Kendall, Distance Correlation, and MIC. For each metric, we find which algorithm gives the highest correlation value. Then, for each application, we calculate the proportion of metrics for which each algorithm performs best, averaging this proportion across all nodes and application replicas. In this way, we estimate the importance of each correlation algorithm for each application.

The results indicate that no single correlation method is optimal for all applications; instead, each application exhibits a distinct pattern reflecting its unique resource usage and runtime behavior. For example, for the *upload* application, Distance Correlation (46.8%) and MIC (32.2%) most frequently identify the strongest relationships, indicating the presence of complex, non-linear dependencies. In contrast, for *gCTF*, Spearman is more effective (45.8%), suggesting predominantly monotonic relationships. Similarly, *Motioncor2*, *ctffind4*, and *FFT mock* all show high proportions for Distance Correlation and MIC, further highlighting the importance of non-linear relationships in these applications. Kendall was not selected as the top method for any metric in any application replica, likely because its correlation values are similar to, but generally lower than, those of Spearman.

5.2. Selected predictor configuration

Fig. 5 shows the proportion of RTT predictor configurations that include the model type, the number of input metrics, and the observation window in all applications and nodes. Each pie chart (from left to right) summarizes the relative frequency of selections for model type, metric count, and observation window size. For clarity, only options representing at least 10% of the total selections are labeled.

The selection of the machine learning model, metric count, and the length of the observation window length varies across applications, highlighting that no single predictor configuration is optimal for all scenarios. Our methodology systematically explores different combinations to identify configurations that balance prediction accuracy with practical constraints, such as maintaining the prediction delay below 10% of the RTT. This approach ensures that predictors are both accurate and efficient in real-world deployments (see Section 3 for details on the selection process).

This process involves a trade-off: using more metrics or longer observation windows can improve prediction accuracy by providing richer system information, but also increases computational and data retrieval overhead. In contrast, reducing these parameters minimizes overhead but may affect the quality of the prediction. For instance, XGBoost is frequently selected because it offers a good balance between accuracy and speed. However, other models may be chosen if they provide higher

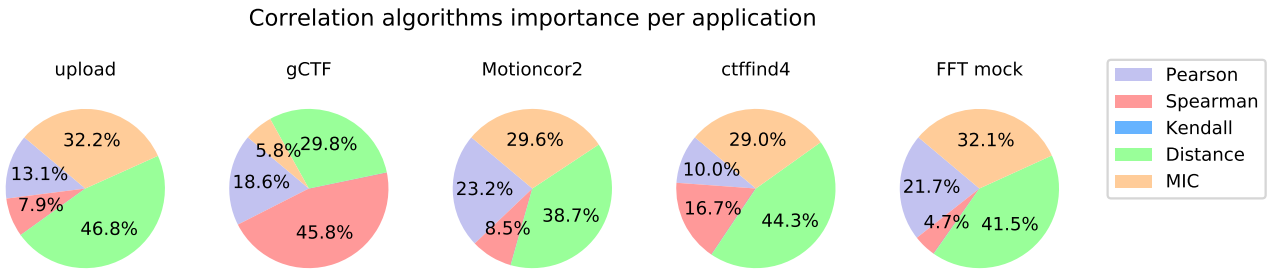


Fig. 4. Proportion of monitoring metrics for each application for which each correlation method yields the highest correlation with RTT.

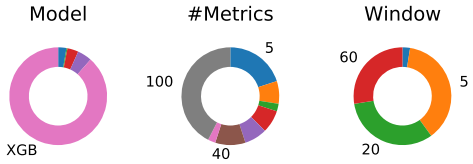


Fig. 5. Proportion of selected RTT predictor configurations by model type, number of metrics, and observation window (in seconds) across all applications and nodes.

accuracy for a specific application or better satisfy strict timing constraints.

5.3. Prediction adaptation on dynamic co-location

To evaluate the accuracy and adaptability of our RTT predictors, we analyzed their RMSE over time under evolving co-location scenarios. Fig. 6 shows how the RMSE evolves for the predictors of each application as they adapt to the changing co-location patterns triggered by the arrival of new data. Each line in the plot corresponds to a predictor instance running on a specific node: there are 8 predictors for upload, ctffind4, and FFT mock, and 3 predictors for gCTF and Motioncor2. If no model satisfies the prediction delay constraint (i.e., inference time below 10% of the RTT), the corresponding line is absent. For example, no valid predictor is deployed for ctffind4 in Worker-4. The x-axis represents the number of hours since the start of the experiment and the y-axis shows the RMSE of each predictor. The cross markers (x) on the lines show the points where the full training of the model is initiated.

We observe several clear patterns in how the RMSE changes over time. The predictors are initialized at different moments, depending on when each application starts running and when sufficient training data become available. In some cases, the RMSE increases after a training (full or re-training). This typically occurs because the new data introduce more complex or harder-to-predict relationships between monitoring metrics and RTT. As the system encounters more diverse co-location scenarios, the RMSE values generally stabilize, indicating that the predictor has learned the dominant interference patterns. After this point, adding more data does not significantly improve the accuracy of the model.

A sharp increase in RMSE (greater than 10%) triggers a full training of the model from scratch. This mechanism appears in many predictors and often leads to improved accuracy. For example, in the FFT mock application running in Worker-1, the RMSE rises sharply to 550% approximately 90 hours after the experiment begins, likely due to abrupt changes in resource usage by other applications. After full training, the new model reduces the error back to the previous levels.

However, there are exceptions. For example, the gCTF predictor in Worker-2 initiates full training several times in a short period but sees little or no improvement in RMSE. This indicates that the lack of improvement is not due to model degradation, but rather to increased complexity and variability in the co-location scenarios, which make RTT prediction inherently more difficult. In these cases, even full training does not yield a better model because the limitation lies in the unpredictable or highly variable nature of the data itself, rather than in the model.

The chosen retraining threshold ($\theta = 10\%$) generally achieved a good trade-off between adaptability and stability. For *the FFT mock* and *Motioncor2*, this value correctly triggered retraining only when RMSE increased dramatically, leading to clear recovery of prediction accuracy. In contrast, for *gCTF* in Worker-2, multiple re-trainings occurred within short intervals without noticeable improvement, suggesting that the threshold was too sensitive for highly variable workloads. In such cases, a slightly higher value (e.g. 15–20%) would reduce redundant retraining, whereas for more stable applications such as *Upload* or *ctffind4*, a lower threshold (5-8%) could enable faster adaptation to subtle shifts. Overall, the 10% setting performs well in all applications but should be tuned according to the application behavior and workload variability.

Table 4
RMSE (%) for all app/node pairs at the final workload stage. The first column shows the worker node IDs. Missing values ("–") indicate that no efficient predictor was found.

	Upload	gCTF	Motioncor2	ctffind4	FFT mock
1	16.52	18.02	15.86	18.26	12.29
2	11.64	9.59	12.41	17.10	22.74
3	20.27	7.75	8.74	10.57	16.22
4	21.35	–	–	–	14.89
5	11.66	–	–	13.97	17.44
6	14.38	–	–	13.03	14.66
7	18.46	–	–	15.14	7.54
8	18.19	–	–	24.79	5.18

Table 4 reports the RMSE of each RTT predictor at the end of the experiment, in all application and node pairs. The results indicate that RTT predictors generally achieve low to moderate error rates for a diverse set of applications and nodes. Applications such as *upload*, *gCTF*, and *Motioncor2* consistently yield RMSE values below 20%, demonstrating strong prediction accuracy. In particular, *gCTF* achieves an RMSE of 7.75% on Worker-3 and remains below 10% on Worker-2. Similarly, *Motioncor2* achieves its lowest RMSE in Worker-3 (8.74%) and performs well on other nodes. The *upload* application shows RMSE values ranging from 11.64% to 21.35%, indicating moderate prediction errors in all nodes.

In contrast, RMSE for *ctffind4* and *FFT mock* exhibit greater variability between nodes. For example, *ctffind4* reaches an RMSE of 24.79% on Worker-8, while *FFT mock* peaks at 22.74% on Worker-2. Despite these exceptions, most RMSE values remain below 20%, confirming that the proposed predictors generalize well in different workloads and deployment environments, even in the presence of application and node-specific differences. A more detailed analysis of how these errors impact load balance decisions is provided in Section 6.

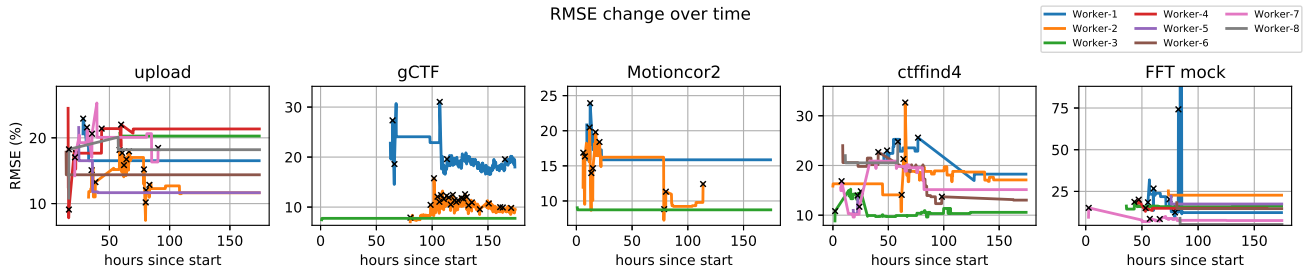


Fig. 6. Evolution of RMSE over time for RTT predictors under changing co-location conditions. Each line represents a predictor deployed on a specific node. Predictors are initialized once enough training data are available, and full training events are marked along the lines.

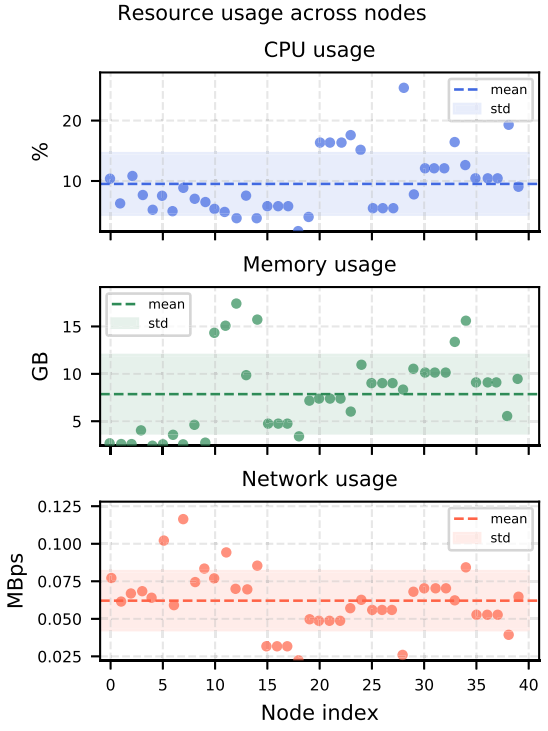


Fig. 7. CPU, memory and network consumption of predictors across nodes and applications.

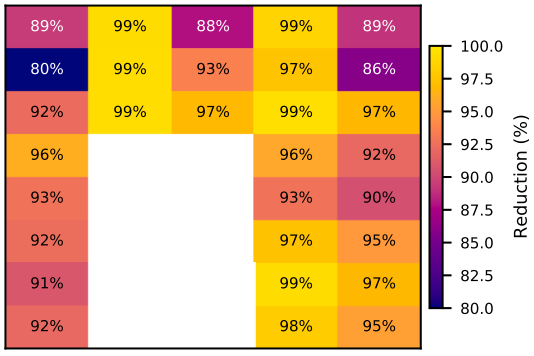


Fig. 8. Dataset size reduction using dynamic bin-based sampling for each application and node pair. Each cell shows the percentage of original samples removed while maintaining RTT distribution coverage.

5.4. Resource consumption of predictors

The developed prediction modules introduce resource overhead due to data collection, correlation analysis, model training, and inference. Therefore, understanding the consumption of running resources is crucial to assess their practical feasibility. Predictors that consume excessive resources can limit the capacity available to co-located applications, reducing their suitability for deployment, even if they deliver high accuracy.

Fig. 7 presents the resource footprint of the predictor processes at all nodes and applications. For each predictor instance, resource usage is calculated as the average consumption measured during its operation. The figure consists of three panels, each displaying the mean and standard deviation of one resource type: CPU usage (in %), memory usage (in GB), and network usage (in Mbps). The x-axis represents the index of each predictor instance, allowing comparison across all predictor deployments.

We observe the following:

- **CPU Usage:** Predictor processes show consistently low CPU utilization, with most instances averaging below 10% and the standard deviation remaining small. The maximum CPU usage does not exceed 25%, indicating a low computational burden. As described in Section 4, predictor processes were restricted to a single CPU core. If this restriction were relaxed, CPU usage could increase, especially for operations such as feature extraction that can benefit from parallel processing.
- **Memory Usage:** Memory consumption varies between predictors, ranging from approximately 2 GB to 18 GB. This variability is due to differences in the size of observation windows, the number of monitoring metrics stored, and the RTTs of the application. Despite these differences, all instances remain within the memory limits of modern edge nodes. The memory requirements are mainly controlled by the dataset balancing process, which limits the dataset size and helps keep memory usage low.
- **Network Usage:** Network overhead is extremely low, with mean usage consistently below 0.08 Mbps and negligible standard deviation. This efficiency is mainly achieved by deploying a local monitoring server on each node, which eliminates the need to retrieve monitoring metrics over the network. Additionally, transmitting predictions requires minimal bandwidth.

Overall, the predictors demonstrate a lightweight resource profile. The low CPU and network demands minimize interference with co-located applications, while memory usage, although variable, remains within feasible bounds. These characteristics make the proposed prediction approach well-suited for deployment in resource-constrained edge environments.

The storage overhead of the prediction workflow is primarily determined by the size of the collected dataset, which consists of the RTT values and the corresponding monitoring metrics from the historical tasks

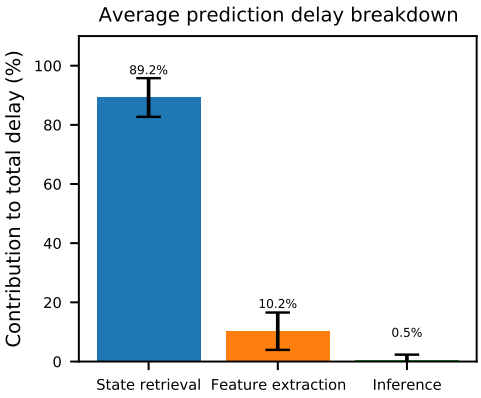


Fig. 9. Contribution of state retrieval, feature extraction, and inference to the total prediction time.

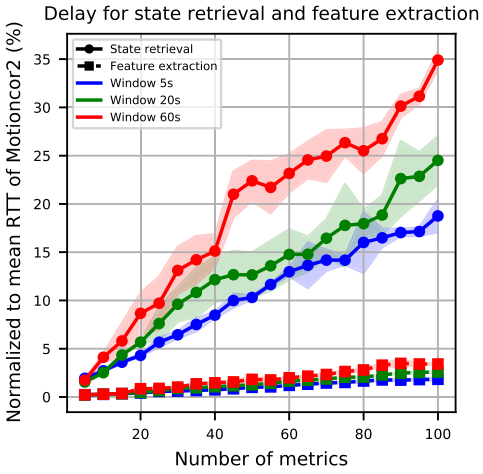


Fig. 10. State retrieval and feature extraction overhead for different observation windows and number of metrics. The delays are normalized to the mean RTT of Motioncor2 in Worker-3.

of each application. In our experiments, the final dataset sizes ranged from 381 to 9493 samples. To minimize storage requirements, our aim is to maintain a uniform RTT distribution in each dataset, as explained in Section 3.

Fig. 8 illustrates the effectiveness of our dataset reduction method. The plot presents the percentage of samples removed across various application and node combinations, with reduction rates typically between 85% and 99%. Although predictors were not trained on the full original datasets, due to their dataset size and memory constraints on small nodes, models built from the reduced datasets offer a favorable balance between prediction accuracy and resource overhead. Using complete datasets would significantly increase both memory usage and training time, making them impractical for our environment. The dynamic sampling method compacts the dataset by eliminating redundant RTT values while preserving less common ones, thus maintaining RTT diversity essential for effective predictor training without unnecessary dataset growth.

5.5. Prediction time breakdown

The total prediction time is critical for maintaining low RTT and consistent performance in time-sensitive applications. When a prediction is triggered, the overall delay consists of three main stages: *state retrieval*, *feature extraction*, and *inference*. Fig. 9 shows the average percentage contribution of each stage to the total prediction time in all pairs of application-nodes, with values normalized to the RTT for direct comparison.

The results indicate that state retrieval is the dominant contributor, accounting for 89.2% of the total delay. Feature extraction adds 10.2%, while inference time is minimal at 0.5%. This reveals that the primary bottleneck lies in monitoring data retrieval (e.g., via Prometheus). Achieving millisecond-scale RTT predictions will therefore require a much faster monitoring system. From the measured delay breakdown (≈99% preparation and 0.5% inference), we assigned $\tau_{\text{prepare}} = 9\%$ and $\tau_{\text{inference}} = 1\%$ to keep the total prediction delay below 10% of RTT. These values reflect the observed delay ratio and can be adjusted in other systems if the relative proportions differ.

Although Prometheus is deployed locally on each node, retrieval delays remain high. Further reductions may be achieved by using alternative monitoring systems or by accessing metrics directly from exporters. Alternative systems for faster metric access include VictoriaMetrics [64] and InfluxDB [65] (with Telegraf as an external collector). This limitation stems from the design of Prometheus, which is optimized for system-wide visibility rather than real-time, sub-second data retrieval. However, addressing this bottleneck falls within the domain of monitoring system optimization rather than the design of the RTT predictor itself.

5.6. State retrieval and feature extraction overhead

Since the total prediction time is mainly dominated by the *state retrieval* and *feature extraction* stages, we further analyze how these delays vary with different system state sizes. The state size is defined by the length of the observation window and the number of monitoring metrics. Fig. 10 illustrates the delay introduced by *state retrieval* (from Prometheus) and *feature extraction* (using `tsfresh`). These delays are presented for the case of *Motioncor2* running on Worker-3 to make it easier to interpret the results and compare different combinations of state and observation window.

We evaluate three observation window sizes, 5, 20 and 60 s, and different numbers of input metrics, ranging from 5 to 100. Each setup is executed five times to capture the variability in execution time, while avoiding excessive load on the monitoring system. The x-axis represents the number of metrics, and the y-axis shows the delay as a percentage of the mean RTT of Motioncor2 on Worker-3. Solid lines indicate the average delay, and shaded regions represent the standard deviation.

The results reveal distinct trends between the two stages. Feature extraction consistently incurs a low delay for all selected observation windows, gradually increasing with the number of monitoring metrics but remaining below 5% of the RTT even in the worst case. In contrast, the state retrieval delay increases significantly with both the number of metrics and the length of the observation window. For a 5 s observation window, the state retrieval delay remains below 20% of the RTT, even when using 100 metrics. However, for a 60 s window, the delay increases sharply, reaching 35% of the RTT at 100 metrics. These results show that feature extraction is efficient and continues to perform well even as the number of metrics and the length of observation windows increases. In contrast, state retrieval becomes the main source of delay when using a larger number of metrics or longer observation windows.

This analysis demonstrates a clear trade-off between the lengths of the observation window, the number of monitoring metrics, and the total prediction delay. Shorter observation windows and fewer metrics reduce overhead, making them preferable for real-time applications. In contrast, longer windows and larger metric sets provide more context but introduce significant delays, mainly due to the time required to query Prometheus. These findings further confirm that Prometheus is the dominant bottleneck in the prediction pipeline, especially for configurations that involve high-volume metric collection over extended pe-

Table 5
Coefficient of variation (CoV) of application RTT for all application–node pairs, comparing scenarios with and without predictor deployment.

Node	upload		gCTF		Motioncor2		ctffind4		FFT mock	
	With	Without	With	Without	With	Without	With	Without	With	Without
Worker-1	37.92%	36.43%	48.64%	50.81%	43.61%	41.70%	70.23%	17.04%	3.78%	3.82%
Worker-2	49.51%	52.04%	45.11%	36.62%	48.15%	49.52%	27.18%	18.53%	8.84%	9.12%
Worker-3	21.31%	21.46%	20.33%	9.86%	32.39%	14.36%	9.37%	9.47%	8.47%	8.38%
Worker-4	21.74%	17.88%	–	–	–	–	23.41%	15.54%	5.56%	3.48%
Worker-5	21.62%	22.94%	–	–	–	–	28.61%	8.97%	5.40%	4.40%
Worker-6	17.27%	16.93%	–	–	–	–	9.63%	8.47%	7.27%	6.79%
Worker-7	19.85%	21.20%	–	–	–	–	8.66%	8.35%	9.53%	11.32%
Worker-8	20.82%	20.58%	–	–	–	–	8.14%	8.30%	9.36%	7.93%

riods. Consequently, some setups become impractical for real-time use due to high retrieval delays. For example, no machine learning model was trained for *ctffind4* in *Worker-4*, as the prediction delay exceeded the acceptable threshold for that application.

5.7. Predictor deployment and RTT variability

We evaluate the impact of running RTT predictors alongside target applications on the same worker nodes, where both share CPU and memory resources. To quantify this effect, we repeat each experiment twice: once with predictors active and once without, using identical co-location scenarios and experiment durations. We then compare the RTT of the application tasks between the two cases. Table 5 reports the Coefficient of Variation (CoV) for each application/node pair, which quantifies the variability of RTT relative to its mean and is defined as:

$$CoV_{\mathrm{RTT}} = \frac{\sigma_{\mathrm{RTT}}}{\mu_{\mathrm{RTT}}} \tag{9}$$

where σ_{RTT} and μ_{RTT} denote the standard deviation and mean RTT, respectively. A higher CoV indicates greater performance variability.

The results reveal mixed effects on the performance of the applications. In some cases, the deployment of predictors increases the variability of the RTT, while in others the CoV remains stable or decreases slightly. For example, on Worker-1, the CoV for *ctffind4* increases sharply from 17.04% to 70.23%, indicating significant resource contention introduced by the predictor. In contrast, *FFT mock* on the same node shows negligible change (3.78% vs. 3.82%). In Worker-2, *ctffind4* experiences higher variability (27.18% vs. 18.53%), while *FFT mock* sees a slight reduction (8.84% vs. 9.12%). Worker-3 exhibits notable increases in CoV for applications *Motioncor2* (32.39% vs. 14.36%) and *gCTF* (20.33% vs. 9.86%). In contrast, nodes such as Worker-7 and Worker-8 display minimal changes.

These differences are primarily attributed to the additional usage of resources by the predictors. Although operations like feature extraction and model training do not run often, they can occasionally cause short bursts of high CPU and memory usage. During these bursts, especially when the node is already running multiple applications, less CPU and memory are available for the main application, which can temporarily affect its performance. Even minor increases in resource usage can affect sensitive applications. Occasional decreases in CoV when predictors are present are likely due to random fluctuations or measurement noise, rather than a true performance improvement.

Overall, running predictors on the same node as applications can introduce additional runtime variability, with the extent depending on the specific node and application sensitivity to resource contention. The predictors are designed to capture their own resource usage through monitoring metrics, allowing them to maintain valid RTT predictions even during concurrent operation. However, for production environments that require minimal performance variability, alternative deployment strategies may be needed. These strategies include offloading predictors to dedicated nodes, scheduling intensive predictor tasks during low-load periods, or reserving extra CPU and memory on shared nodes. Reserving resources guaranties that the application has exclusive access to a portion of the system capacity, thereby minimizing performance drops caused by competition with predictor processes.

6. Impact on load-balancing

To analyze the impact of performance-aware load balancing compared to other strategies in edge computing environments, we developed a simulation framework that models a cluster of heterogeneous nodes running multiple application replicas. The simulation evaluates how placement decisions affect RTT and resource utilization under varying hardware configurations and co-location scenarios. All reported results are averaged over 200 independent trials to ensure statistical reliability.

6.1. Simulation workflow

Each simulation trial begins by initializing a set of compute nodes (for example, 10 nodes), where each node is assigned a number of CPU cores and a memory capacity randomly selected from predefined ranges to reflect hardware heterogeneity. To capture additional variation in hardware performance, each node is also assigned an acceleration factor. Applications are characterized by predefined mean RTT, resource requirements, and sensitivity to performance interference. All simulation parameters are derived from measurements collected on a real-world Kubernetes cluster running SPA workloads. Due to space constraints, these parameters are not shown here, but are available in the public repository (see the corresponding section). The goal of the simulation is to emulate realistic system dynamics based on these measurement-driven parameters, rather than to predict absolute performance values.

Interference effects are modeled using an empirically derived interference matrix, which quantifies the increase in RTT when applications are co-located on the same node. The placement of replicas across nodes is randomized in each trial to isolate the impact of the load-balancing strategy.

For each request, the RTT is computed as:

$$\mathrm{RTT}_{\mathrm{actual}} = x \cdot (1 + \alpha), \quad \text{where } x \sim \mathrm{LogNormal}(\mu, \sigma), \tag{10}$$

where α is the acceleration factor. The log-normal parameters μ and σ are derived using the application mean RTT $\bar{r}$ (fixed for each application) and the standard deviation s, which is determined for each co-location scenario using an empirically constructed interference matrix. The interference matrix quantifies the increase in RTT variability due to resource contention between different applications running on the same node. The log-normal distribution was selected based on RTT measurements from our experiments. The values of μ and σ are given by:

$$\mu = \log\left(\frac{\bar{r}^2}{\sqrt{s^2 + \bar{r}^2}}\right), \qquad \sigma = \sqrt{\log\left(1 + \frac{s^2}{\bar{r}^2}\right)} \tag{11}$$

Three load balancing strategies are considered:

- **Round Robin:** Replicas are selected in sequential order if idle.

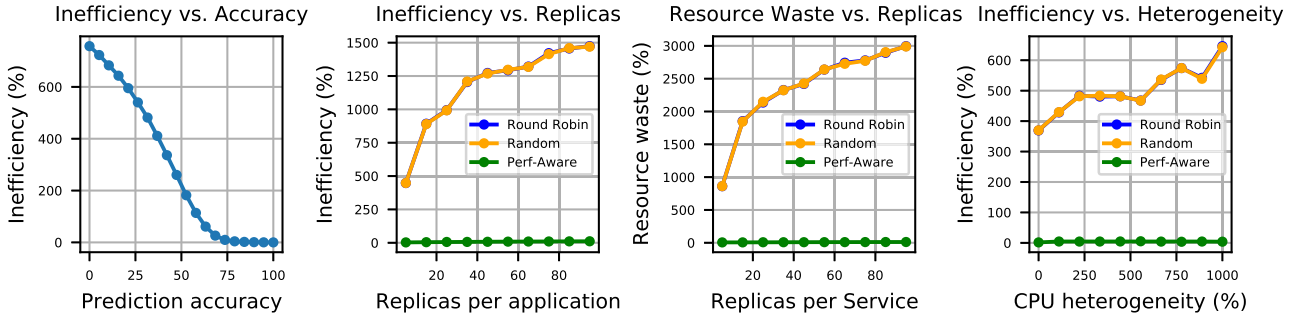


Fig. 11. Simulation results of RTT predictor integration into load balancing.

- **Random Selection:** Replicas are chosen randomly from the available idle ones.
- **Performance-Aware:** Idle replicas are evaluated based on predicted RTTs. Each predicted RTT is computed as a noisy estimate of the true RTT:

$$\text{RTT}_{\text{predicted}} = \text{RTT}_{\text{actual}} + \mathcal{N}(0, \epsilon), \tag{12}$$

where $\epsilon = (1 - p) \cdot \text{RTT}_{\text{actual}}$ and $p \in [0, 1]$ is the prediction accuracy. The replica with the lowest predicted RTT is selected.

Requests are scheduled according to the chosen strategy, and each replica maintains a local busy-until timestamp to enforce concurrency limits. For each request, the total CPU and memory used by a replica is calculated by multiplying its per-request resource requirement by the time it takes to finish.

6.2. Expected gain in load-balancing

We compare the performance-aware load balancing strategy with traditional methods such as round-robin and random assignment. The evaluation focuses on four key factors: prediction accuracy, number of replicas, resource waste, and system heterogeneity. To quantify the effectiveness of the load balancer, we use *scheduling inefficiency*, defined as the performance loss (in terms of RTT or resource usage) relative to an ideal load balancer with perfect knowledge of system performance.

The effectiveness of the performance-aware load balancer depends on the accuracy of its performance predictions. The first subplot of Fig. 11 illustrates how scheduling inefficiency changes with prediction accuracy. As prediction accuracy increases, scheduling inefficiency decreases, dropping to nearly zero when accuracy reaches 80%. This shows that a near-perfect prediction is not required for efficient load balancing. Beyond this threshold, further increases in accuracy do not lead to further gains, as the load balancer is already able to consistently select faster application replicas and avoid slower ones. In contrast, lower prediction accuracy leads to more suboptimal placement decisions and higher inefficiency. The accuracy threshold for optimal performance may vary depending on the characteristics of the cluster, such as hardware heterogeneity. For all subsequent simulation results, we assume that the predictors achieve 80% accuracy.

The second subplot of Fig. 11 illustrates how scheduling inefficiency changes with the number of replicas per application for different load balancing strategies. As the number of replicas increases, the performance-aware load balancer maintains low inefficiency, while round-robin and random strategies exhibit increasing inefficiency. This shows that performance-aware load balancing scales effectively by leveraging RTT predictions for task placement. In contrast, baseline strategies ignore application interference, resulting in more suboptimal placements as the number of replicas and possible placement options increases.

In resource-constrained environments, it is essential to minimize both RTT and overall resource consumption. The decisions made by the load balancer directly affect how efficiently cluster resources are used. The third subplot of Fig. 11 shows resource waste as the number of replicas increases for each strategy, where resource waste refers to the extra resources consumed compared to an ideal load balancer with perfect knowledge. For both round-robin and random strategies, resource waste increases as more replicas are added, since these approaches do not account for current system state or performance and often lead to inefficient placements. In contrast, the performance-aware load balancer keeps resource waste significantly lower and scales more efficiently by using RTT predictions to select suitable replicas and avoid unnecessary contention. This results in better resource utilization, as tasks are completed faster and resources are released more quickly, preventing both over-provisioning and under-utilization. The simulation also accounts for any additional consumption from the RTT predictor itself.

Resource clusters typically include nodes with varying hardware and software configurations, leading to different RTTs for the same task across nodes. Hardware heterogeneity therefore plays a key role in load-balancing effectiveness. The fourth subplot of Fig. 11 shows that scheduling inefficiency increases as the heterogeneity of the CPU increases. Traditional strategies like round-robin and random assignment are less effective in this context, as they overlook node performance differences. In contrast, the performance-aware load balancer adapts to resource variability, maintaining low inefficiency even as heterogeneity grows. Using RTT predictors, it makes informed placement decisions that take into account hardware differences, resulting in more efficient and predictable task execution.

Overall, the simulation results demonstrate that performance-based load balancing, guided by RTT prediction, offers substantial gains in efficiency, scalability, and adaptability, especially in heterogeneous and dynamic environments. These advantages make it an attractive solution for deployment in modern cloud and edge-based systems.

7. Lessons learned

This section summarizes the main insights from developing and evaluating the proposed performance prediction approach, emphasizing practical implications for edge computing environments:

1. **Diverse correlation methods:** Both linear and non-linear dependencies exist between RTT and monitoring metrics. Using multiple correlation techniques ensures robust identification of key metrics and resilient predictor design.
2. **Configuration sensitivity:** No single configuration (model type, metric count, observation window) performs best universally. Each application-node pair requires tailored parameters to balance accuracy and computational cost.
3. **Adaptation and retraining:** Predictors adapt to evolving co-location patterns, with re-training stabilizing RMSE as data accumulate. The full training trigger threshold ($\theta = 10\%$) balances responsiveness and stability for most applications, though workloads

with highly fluctuating error patterns (e.g. *gCTF*) may benefit from a higher threshold to avoid repeated full trainings, while more stable cases can tolerate a lower one.

4. **Predictor overhead:** Predictors introduce minimal system load, with low CPU and network usage, moderate memory needs, and reduced storage due to balanced datasets–making them suitable for runtime edge deployment.
5. **Prediction delay:** RTT prediction delay is dominated by monitoring data retrieval rather than inference, justifying the 10% delay allocation ($\tau_{\text{prepare}} = 9\%$, $\tau_{\text{inference}} = 1\%$). Optimizing monitoring efficiency can further reduce τ_{prepare} and enable faster end-to-end prediction.
6. **State size and responsiveness:** Larger observation windows or excessive metrics increase retrieval time and reduce responsiveness. Selecting compact yet informative state representations is critical for low-latency prediction.
7. **Interference and variability:** Running predictors alongside applications can slightly increase variability, depending on workload and node conditions. This can be mitigated by isolating predictors or scheduling training during idle periods.
8. **Accuracy threshold and scalability:** Once the prediction accuracy exceeds a practical threshold, further gains have negligible impact on load-balancing efficiency. Performance-aware strategies remain effective under interference and heterogeneity, minimizing resource waste and supporting scalable operation.

8. Conclusions and future work

We propose Morpheus, a practical framework for building RTT predictors using the historical state of monitoring data. Instead of arbitrarily selecting input features, our approach leverages the previously developed perfCorrelate framework to identify the most relevant monitoring metrics. The resulting predictors run in parallel with applications, providing fast predictions with minimal resource overhead while maintaining high accuracy under diverse runtime conditions, including resource contention and hardware heterogeneity. This is achieved by carefully balancing the construction of the dataset, the length of the observation window, the selection of features and the complexity of the model, all guided by the correlation between the monitoring metrics and the RTT.

To evaluate the impact of prediction-aware load balancing, we simulate a realistic edge computing environment where RTT predictors are integrated into the request routing process. Our results show that performance-aware load balancing driven by accurate RTT predictions significantly improves application performance, reduces resource waste, and enhances robustness in heterogeneous systems. In addition, we outline key lessons learned throughout the development and evaluation process to guide future improvements.

In future work, we plan to integrate RTT predictors into a production-grade load balancer to evaluate their impact on real clusters. We will also examine alternative designs to further reduce the prediction delay without sacrificing accuracy and benchmark our approach against state-of-the-art load-balancing methods. In addition, we will sweep the threshold parameters to assess their impact and automate their value selection.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the authors used ChatGPT in order to improve the clarity of certain sentences of the manuscript. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

CRediT authorship contribution statement

Panagiotis Giannakopoulos: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Bart van Knippenberg:** Writing – review & editing, Writing – original draft, Software, Resources; **Kishor Chandra Joshi:** Writing – review & editing, Supervision; **Nicola Calabretta:** Writing – review & editing, Supervision, Conceptualization; **George Exarchakos:** Writing – review & editing, Supervision, Resources, Project administration, Funding acquisition.

Data availability

The datasets and source code used in this study are publicly available on Zenodo. This includes all scripts, processed datasets, and instructions to reproduce the experiments and figures presented in the paper. Link to repository: https://doi.org/10.5281/zenodo.18848045

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests:

Panagiotis Giannakopoulos reports financial support and writing assistance were provided by Thermo Fisher Scientific Inc Pittsburgh. Georgios Exarchakos reports financial support, administrative support, and travel were provided by Dutch Research Council. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This publication is part of the project ADAPTOR: Autonomous Distribution Architecture on Progressing Topologies and Optimization of Resources (with project number 18651 of the research Open Technology Programme which is (partly) financed by the Dutch Research Council (NWO). The image data used for the experiments was acquired at Thermo Fisher Scientific (Eindhoven, The Netherlands). The computational experiments, including execution of the described algorithms, were executed at the Eindhoven University of Technology (Eindhoven, The Netherlands).

References

[1] M. Polese, L. Bonati, S. D'Oro, S. Basagni, T. Melodia, Understanding O-RAN: architecture, interfaces, algorithms, security, and research challenges 25 (2) 1376–1411. https://doi.org/10.1109/COMST.2023.3239220

[2] B. Wydrowski, R. Kleinberg, S.M. Rumble, A. Archer, Load is not what you should balance: introducing prequal, 2024, pp. 1285–1299.

[3] A. Kuriata, R.G. Illikkal, Predictable performance for QoS-Sensitive, scalable, multitenant function-as-a-Service deployments, in: M. Paasivaara, P. Kruchten (Eds.), Agile Processes in Software Engineering and Extreme Programming – Workshops, 396, Springer International Publishing, pp. 133–140. Series Title: Lecture Notes in Business Information Processing. https://doi.org/10.1007/978-3-030-58858-8_14

[4] A. Maricq, D. Duplyakin, I. Jimenez, C. Maltzahn, R. Stutsman, R. Ricci, Taming performance variability (2018) 409–425.

[5] D. Duplyakin, A. Uta, A. Maricq, R. Ricci, et al., On studying CPU performance of CloudLab hardware, in: 2019IEEE 27th International Conference on Network Protocols (ICNP), IEEE, 2019, pp. 1–2. https://doi.org/10.1109/ICNP.2019.8888128

[6] Y. Fu, S. Zhang, J. Terrero, Y. Mao, G. Liu, S. Li, D. Tao, et al., Progress-based container scheduling for short-lived applications in a Kubernetes cluster, in: 2019IEEE International Conference on Big Data (Big Data), IEEE, 2019, pp. 278–287. https://doi.org/10.1109/BigData47090.2019.9006427

[7] P. Townend, S. Clement, D. Burdett, R. Yang, J. Shaw, B. Slater, J. Xu, Invited paper: improving data center efficiency through holistic scheduling in Kubernetes, in: 2019IEEE International Conference on Service-Oriented System Engineering (SOSE), IEEE, 2019, pp. 156–15610. https://doi.org/10.1109/SOSE.2019.00030

[8] H. Mohamed, O. El-Gayar, End-to-End Latency Prediction of Microservices Workflow on Kubernetes: A Comparative Evaluation of Machine Learning Models and Resource Metrics, 2021.

[9] Y. Wang, V. Lee, G.-Y. Wei, D. Brooks, Predicting new workload or CPU performance by analyzing public datasets 15 (4) (2018) 1–21. https://doi.org/10.1145/3284127

[10] T. Hoefler, R. Belli, Scientific benchmarking of parallel computing systems: twelve ways to tell the masses when reporting performance results, in: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC '15, Association for Computing Machinery, New York, NY, USA, 2015. https://doi.org/10.1145/2807591.2807644

[11] H. Ballani, P. Costa, T. Karagiannis, A. Rowstron, Towards predictable datacenter networks, in: Proceedings of the ACM SIGCOMM 2011 Conference, 2011, pp. 242–253.

[12] S. Chunduri, K. Harms, S. Parker, V. Morozov, S. Oshin, N. Cherukuri, K. Kumaran, Run-to-run variability on Xeon Phi based cray XC systems, in: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, ACM, 2017, pp. 1–13. https://doi.org/10.1145/3126908.3126926

[13] L. Traini, D. Di Pompeo, M. Tucci, B. Lin, S. Scalabrino, G. Bavota, M. Lanza, R. Oliveto, V. Cortellessa, How software refactoring impacts execution time 31 (2) (2021) 25:1–25:23. https://doi.org/10.1145/3485136

[14] C. Laaber, T. Yue, S. Ali, Evaluating search-based software microbenchmark prioritization (1) (2024) 1–16. https://doi.org/10.1109/TSE.2024.3380836

[15] D.G. Reichelt, S. Kühne, W. Hasselbring, Automated identification of performance changes at code level, in: 2022IEEE 22nd International Conference on Software Quality, Reliability and Security (QRS), 2022, pp. 916–925. ISSN: 2693–9177. https://doi.org/10.1109/QRS57517.2022.00096

[16] N. Sukhija, E. Bautista, O. James, D. Gens, S. Deng, Y. Lam, T. Quan, B. Lalli, Event management and monitoring framework for HPC environments using ServiceNow and prometheus, in: Proceedings of the 12th International Conference on Management of Digital EcoSystems, MEDES '20, Association for Computing Machinery, 2020, pp. 149–156. https://doi.org/10.1145/3415958.3433046

[17] O. Mart, C. Negru, F. Pop, A. Castiglione, Observability in Kubernetes cluster: automatic anomalies detection using prometheus, in: 2020IEEE 22nd International Conference on High Performance Computing and Communications; IEEE 18th International Conference on Smart City; IEEE 6th International Conference on Data Science and Systems (HPCC/SmartCity/DSS), 2020, pp. 565–570. https://doi.org/10.1109/HPCC-SmartCity-DSS50907.2020.00071

[18] J. Qiu, Q. Du, K. Yin, S.-L. Zhang, C. Qian, A causality mining and knowledge graph based method of root cause diagnosis for performance anomaly in cloud applications 10 (6) (2020) 2166. https://doi.org/10.3390/app10062166

[19] L. Meng, F. Ji, Y. Sun, T. Wang, et al., Detecting anomalies in microservices with execution trace comparison 116 (2021) 291–301. https://doi.org/10.1016/j.future.2020.10.040

[20] L. Wu, J. Tordsson, J. Bogatinovski, E. Elmroth, O. Kao, et al., MicroDiag: fine-grained performance diagnosis for microservice systems, in: 2021IEEE/ACM International Workshop on Cloud Intelligence (CloudIntelligence), 2021, pp. 31–36. https://doi.org/10.1109/CloudIntelligence52565.2021.00015

[21] Y. Zhao, D. Duplyakin, R. Ricci, A. Uta, et al., Cloud performance variability prediction, in: Companion of the ACM/SPEC International Conference on Performance Engineering, ACM, 2021, pp. 35–40. https://doi.org/10.1145/3447545.3451182

[22] S. Lundberg, S.-I. Lee, A Unified Approach to Interpreting Model Predictions, 2017. 1705.07874 [cs, stat]arXiv:1705.07874

[23] P. Giannakopoulos, B. van Knippenberg, K.C. Joshi, N. Calabretta, G. Exarchakos, PerfCorrelate: performance variability correlation framework 170 (2025) 107827. https://doi.org/10.1016/j.future.2025.107827

[24] C. Delimitrou, C. Kozyrakis, Quasar: resource-efficient and QoS-aware cluster management, in: Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS '14, Association for Computing Machinery, 2014, pp. 127–144. https://doi.org/10.1145/2541940.2541941

[25] D. Dauwe, E. Jonardi, R.D. Friese, S. Pasricha, A.A. Maciejewski, D.A. Bader, H.J. Siegel, HPC node performance and energy modeling with the co-location of applications 72 (12) (2016) 4771–4809. https://doi.org/10.1007/s11227-016-1783-y

[26] C. Li, J. Li, Y. Gao, J. Song, Z. Wang, Multidimensional resource load-aware task migration in mobile edge computing 175 (2026) 108091. https://doi.org/10.1016/j.future.2025.108091

[27] Y. Gao, L. Wang, Z. Xie, Z. Qi, J. Zhou, Energy- and quality of experience-aware dynamic resource allocation for massively multiplayer online games in heterogeneous cloud computing systems 16 (3) (2023) 1793–1806. https://doi.org/10.1109/TSC.2022.3190447

[28] B. Alankar, G. Sharma, H. Kaur, R. Valverde, V. Chang, Experimental setup for investigating the efficient load balancing algorithms on virtual cloud 20 (24) (2020) 7342. https://doi.org/10.3390/s20247342

[29] L. Laukka, C. Fransson, N. Pappas, Load balancing traffic among Kubernetes replicas by utilizing workload estimation, in: 2023IEEE Conference on Standards for Communications and Networking (CSCN), 2023, pp. 353–356. ISSN: 2644–3252. https://doi.org/10.1109/CSCN60443.2023.10453145

[30] J. Chen, Y. Yang, A context-aware approach to task scheduling for time series data prediction in mobile edge computing 28 (1) (2023) 421–431. https://doi.org/10.1007/s11036-023-02131-9

[31] O. Michaelis, S. Schmid, H. Mostafaei, L3: latency-aware load balancing in multi-cluster service mesh, in: Proceedings of the 25th International Middleware Conference, Middleware '24, Association for Computing Machinery, 2024, pp. 49–61. https://doi.org/10.1145/3652892.3654793

[32] M.S. Aslanpour, A.N. Toosi, M.A. Cheema, M.B. Chhetri, M.A. Salehi, Load balancing for heterogeneous serverless edge computing: a performance-driven and empirical approach 154 (2024) 266–280. https://doi.org/10.1016/j.future.2024.01.020

[33] R. Bhattacharya, T. Wood, BLOC: balancing load with overload control in the microservices architecture, in: 2022IEEE International Conference on Autonomic Computing and Self-Organizing Systems (ACSOS), 2022, pp. 91–100. https://doi.org/10.1109/ACSOS55765.2022.00027

[34] A.B. Kathole, V.K. Singh, A. Goyal, S. Kant, A.S. Savyanavar, S.A. Ubale, P. Jain, M.T. Islam, Novel load balancing mechanism for cloud networks using dilated and attention-based federated learning with coati optimization 15 (1) (2025) 15268. https://doi.org/10.1038/s41598-025-99559-8

[35] J. Hu, Z. Zhou, J. Wang, Hierarchical adaptive learning-Based congestion control with low training overhead for datacenter networks 22 (5) (2025) 4061–4069. https://doi.org/10.1109/TNSM.2025.3589637

[36] J. Hu, W. Luo, A. Tolba, J. Wang, Traffic-aware load balancing based on deep reinforcement learning in cloud-based industrial data centers 21 (12) (2025) 9229–9239. https://doi.org/10.1109/TII.2025.3594051

[37] H. He, E.A. Garcia, Learning from imbalanced data, IEEE Trans. Knowl. Data Eng. 21 (9) (2009) 1263–1284. https://doi.org/10.1109/TKDE.2008.239

[38] D. Freedman, P. Diaconis, On the histogram as a density estimator:L2 theory 57 (4) (1981) 453–476. https://doi.org/10.1007/BF01025868

[39] S.S. Shapiro, M.B. Wilk, An analysis of variance test for normality (complete samples), Biometrika 52 (3/4) (1965) 591–611.

[40] M. Christ, N. Braun, J. Neuffer, A.W. Kempa-Liehr, et al., Time series FeatuRe extraction on basis of scalable hypothesis tests (tsfresh – A Python package) 307 (2018) 72–77. https://doi.org/10.1016/j.neucom.2018.03.067

[41] T. Zhang, Solving large scale linear prediction problems using stochastic gradient descent algorithms, in: Proceedings of the Twenty-First International Conference on Machine Learning, ICML '04, Association for Computing Machinery, New York, NY, USA, 2004, p. 116. https://doi.org/10.1145/1015330.1015332

[42] T. Chen, C. Guestrin, XGBoost: A scalable tree boosting system, in: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '16, Association for Computing Machinery, New York, NY, USA, 2016, p. 785–794. https://doi.org/10.1145/2939672.2939785

[43] T.K. Ho, Random decision forests, in: Proceedings of 3rd International Conference on Document Analysis and Recognition, 1, IEEE, 1995, pp. 278–282.

[44] I. Goodfellow, Y. Bengio, A. Courville, Deep Learning, MIT Press, 2016.

[45] J.J. Hopfield, Neural networks and physical systems with emergent collective computational abilities, Proc. Natl. Acad. Sci. 79 (8) (1982) 2554–2558. https://doi.org/10.1073/pnas.79.8.2554

[46] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Comput. 9 (8) (1997) 1735–1780.

[47] J. Wu, Introduction to convolutional neural networks, Nat. Key Lab Novel Software Technol. Nanjing Univ. China 5 (23) (2017) 495.

[48] J. Chung, C. Gulcehre, K. Cho, Y. Bengio, Empirical evaluation of gated recurrent neural networks on sequence modeling, arXiv preprint arXiv:1412.3555 (2014).

[49] B. Gregorutti, B. Michel, P. Saint-Pierre, Correlation and variable importance in random forests 27 (3) (2017) 659–678. https://doi.org/10.1007/s11222-016-9646-1

[50] J. Bornschein, F. Visin, S. Osindero, Small data, big decisions: model selection in the small-data regime, in: Proceedings of the 37th International Conference on Machine Learning, ICML'20, JMLR.org, 2020.

[51] A.F. Brilot, J.Z. Chen, A. Cheng, J. Pan, S.C. Harrison, C.S. Potter, B. Carragher, R. Henderson, N. Grigorieff, Beam-induced motion of vitrified specimen on holey carbon film, J. Struct. Biol. 177 (3) (2012) 630–637. https://doi.org/10.1016/j.jsb.2012.02.003

[52] S.Q. Zheng, E. Palovcak, J.-P. Armache, K.A. Verba, Y. Cheng, D.A. Agard, MotionCor2: anisotropic correction of beam-induced motion for improved cryo-electron microscopy, Nat. Methods 14 (4) (2017) 331–332. https://doi.org/10.1038/nmeth.4193

[53] R.A. Crowther, R. Henderson, J.M. Smith, MRC image processing programs, J. Struct. Biol. 116 (1) (1996) 9–16. https://doi.org/10.1006/jsbi.1996.0003

[54] K. Zhang, GCTF: real-time CTF determination and correction 193 (1) (2016) 1–12. https://doi.org/10.1016/j.jsb.2015.11.003

[55] A. Rohou, N. Grigorieff, CTFFIND4: fast and accurate defocus estimation from electron micrographs, J. Struct. Biol. 192 (2) (2015) 216–221. Recent Advances in Detector Technologies and Applications for Molecular TEM. https://doi.org/10.1016/j.jsb.2015.08.008

[56] Y. Deng, F. Grollios, H. Kohr, B. van Knippenberg, M. Janus, F. Caglar, Smart EPU: SPA getting intelligent, Microsc. Microanal. 27 (S1) (2021) 454–455. https://doi.org/10.1017/S1431927621002129

[57] Kubernetes, (https://kubernetes.io).

[58] Docker, (https://www.docker.com/).

[59] Containerd, (https://containerd.io/).

[60] Prometheus, (https://prometheus.io/).

[61] Grafana, (https://grafana.com/).

[62] NVIDIA DGCM, (https://docs.nvidia.com/datacenter/cloud-native/gpu-telemetry/dcgm-exporter.html).

[63] jaegertracing, (https://www.jaegertracing.io/).

[64] VictoriaMetrics: Scalable, fast and cost-effective time series database, (https://victoriametrics.com/).

[65] InfluxDB: Time Series Database, (https://www.influxdata.com/).



Panagiotis Giannakopoulos graduated from the Technical University of Crete, Greece with an Integrated Master's degree in Electrical and Computer Engineering in 2020. He started his PhD journey at Eindhoven University of Technology, The Netherlands. His research interests include performance prediction, resource allocation and cloud/edge computing.



Bart van Knippenberg, holding a Master's degree in Medical Engineering from the Eindhoven University of Technology, is leading the Data & AI Infrastructure group within the Materials and Structural Analysis Division at Thermo Fisher Scientific. His research interests include algorithm development for computer vision, image acquisition workflows for Electron Microscopy, and infrastructure for AI development.



Kishor C. Joshi is an Assistant Professor in the Electrical Engineering Department of TU Eindhoven since Sept 2021. Before joining TU/e, he was a postdoctoral researcher at University of Amsterdam (2021–2022) in the Complex Cyber Infrastructure group and in the L2S laboratory of CNRS at CentraleSupelec (2019–2020) in Paris, France. He received his PhD from Delft University of Technology, Netherlands in 2017. He is a recipient of the prestigious Marie-Curie Postdoctoral Fellowship (MSCA-IF) from European Commission in 2017. His research interests are in Resource allocations in the cloud-native implementation of 5G/6G Networks, Millimeter Wave Communications and Tactile Internet.



Nicola Calabretta (Member, IEEE) received the BS and MS degrees in telecommunications engineering from Politecnico di Torino, Turin, Italy, in 1995 and 1999, respectively, and the Ph.D. degree from Eindhoven University of Technology (TU/e), The Netherlands, in 2004. From 2004 to 2007, he was a Researcher with Scuola Superiore Sant'Anna University, Italy. He is currently a Professor with the Institute of Photonic Integration, (TU/e). He has co-authored over 570 publications and holds five patents. His research interests include smart optical data centers and telecom optical networks, optical processing, optical packet switching, and semiconductor optical amplifier based optical switching and processing.



George Exarchakos (Member IEEE) received his BS degree in informatics and telcecommunications from University of Athens in 2004, his MSc degree in advanced computing from the Imperial College London in 2005 and his PhD degree at the University of Surrey in 2009. He is now an Associate Professor at TU/e on in-network intelligence for reliable network architectures, heading the Advanced Networking Lab. He has co-authored a book (Springer, 2011), over 100 publications and edited 4 other books. His research interests are on resource allocation in wired/wireless networks, scheduling, in-network intelligence and network architectures.